

INTERNAL

API Reference

Quenyx vOPS HUB — v1.0.0 RC1 · Document v2.2

Document Version	2.2
Software Version	v1.0.0 RC1
Applies To	Quenyx vOPS HUB v1.0.0 RC1
Classification	Internal
Owner	Platform Engineering
Status	Released
Last Updated	2026-06-30
Document Type	API reference

REVISION HISTORY

Version	Date	Notes
1.0	2026	Initial v1 pack (through Sprint 19).
2.0	2026-06-29	Aligned to v1.0.0 RC1; includes Unified AI Workspace (Sprint 20) endpoints.
2.1	2026-06-30	Added QynSight Operations Intelligence (Sprint 21) endpoints under <code>/api/qynsight/intelligence/*`</code> .
2.2	2026-06-30	Added the AI Adapter discovery API (<code>/api/ai/adapters`</code> , <code>/api/ai/actions`</code>) and QynAsset Asset Intelligence (<code>/api/qynasset/intelligence/*`</code>) — Sprint 22.

Table of Contents

- 1. Authentication
- 2. Common headers
- 3. Error format
- 4. Workspace / project routes
- 5. QynSight APIs (summary) — `project.module:qynsight`
- 6. Compliance corpus APIs
- 7. AI context APIs (consumption contract, no AI execution)
- 8. Graph APIs
- 9. Mapping APIs
- 10. Evidence APIs
- 11. Gap APIs
- 12. Recommendation APIs
- 13. Copilot APIs — throttle compliance-copilot
- 14. Retrieval / RAG APIs
- 15. Executive APIs (read-only) — throttle compliance-executive
- 16. Platform AI capability endpoint
- 17. Quenyx AI (Unified AI Workspace — Sprint 20)
- 18. QynSight Operations Intelligence (Sprint 21)
- 19. QynAsset Asset Intelligence + AI Adapter Platform (Sprint 22)
- 20. Notes on examples

08 — API Reference

Audience: Engineers, integrators. **Source:** Derived from `php artisan route:list` (261 routes) and the `routes/*.php` files at Sprint 19. **No endpoints are invented.** Regenerate this doc when routes change.

projects vs workspaces . Most tenant routes exist under **both** `/api/projects/{project}/...` and `/api/workspaces/{project}/...` — they are **aliases to the same controllers**. Below, paths are shown with `/workspaces/{project}` (the canonical form); swap `workspaces` → `projects` for the alias.

1. Authentication

- **Scheme:** Laravel **Sanctum** bearer tokens.
- **Obtain a token:** `POST /api/auth/login` (`POST /api/auth/register` to create an account).
- **Use it:** `Authorization: Bearer <token>` on every authenticated route.
- **Identity:** `GET /api/auth/me`; `logout POST /api/auth/logout` .
- **Public (no user auth):** `GET /api/health` , agent ingestion endpoints (token/secret auth).

2. Common headers

```
Authorization: Bearer <token>
Accept: application/json
Content-Type: application/json      # for POST/PUT/PATCH
```

3. Error format

Standard Laravel JSON errors:

```
{ "message": "Unauthenticated." }           // 401
{ "message": "This action is unauthorized." } // 403
{ "message": "Not Found" }                 // 404
{ "message": "The given data was invalid.", // 422
  "errors": { "field": ["validation message"] } }
{ "message": "Too Many Attempts." }       // 429 (throttled routes)
```

4. Workspace / project routes

Method	Path	Controller
GET/POST	/api/projects · /api/workspaces	ProjectController@index/sto
GET/PUT/DELETE	/api/workspaces/{project}	ProjectController@show/upda
GET	/api/workspaces/{project}/entitlements	ProjectSubscriptionControl
GET/PUT	/api/workspaces/{project}/subscription	ProjectSubscriptionControl
GET	/api/workspaces/{project}/modules · /modules/access	ProjectModuleController
PUT	/api/workspaces/{project}/modules/{moduleKey}/override	ProjectModuleOverrideContro
GET..DELETE	/api/workspaces/{project}/memberships...	ProjectMembershipController
GET	/api/workspaces/{project}/audit-logs	AuditLogController@index
GET..PUT	/api/workspaces/{project}/integrations...	ProjectIntegrationControlle
GET..POST	/api/workspaces/{project}/billing/...	BillingController
GET	/api/dashboard , /api/modules , /api/plans	platform meta

5. QynSight APIs (summary) — project.module:qynsight

Prefix: /api/workspaces/{project}/observe/...

- **Summary / instances:** summary , instances , instances/summary .
- **Services / checks:** services , service-definitions , run-checks .
- **Performance:** performance/metrics , real-time/metrics , real-time/system-info , real-time/thresholds .
- **Capacity:** capacity-planning , capacity-planning/export , capacity/metrics .
- **Alerts:** alerts/rules (GET/POST/PUT/DELETE/ toggle) , alerts/summary , alerts/history , alerts/events/{event}/acknowledge , alerts/channels , monitoring-profile (GET/PUT).
- **Infrastructure:** infrastructure/topology , infrastructure/connections , infrastructure/port-scans (GET/POST run).
- **Reports / data sources:** reports , reports/summary , data-sources , data-sources/summary .
- **Targets / hosts:** targets (GET/PUT), targets/validate , targets/{hostId}/port-scan .
- **Agents:** /api/workspaces/{project}/agents... (list, metadata, enrollment tokens, revoke, destroy).

Agent ingestion (token-auth, no user): GET /api/agents/download/{platform} , POST /api/agents/register , POST /api/agents/{agent}/heartbeat|metrics|inventory .

6. Compliance corpus APIs

Global (read): `/api/compliance/corpus/frameworks...` Workspace (entitled, `project.qynshield`): `/api/workspaces/{project}/compliance/corpus/frameworks...`

Path (suffix under <code>/corpus</code>)	Returns
<code>frameworks</code>	frameworks list
<code>frameworks/{frameworkKey}/releases</code>	releases
<code>frameworks/{frameworkKey}/releases/{releaseCode}/summary</code>	release summary
<code>.../domains</code> · <code>.../domains/{domainCode}</code>	domains
<code>.../controls/{controlCode}</code>	control detail
<code>.../search</code>	corpus search

7. AI context APIs (consumption contract, no AI execution)

`/api/workspaces/{project}/compliance/ai-context/frameworks/{frameworkKey}/releases/{releaseCode}/...` → `summary`, `domains/{domainCode}`, `controls/{controlCode}`, `search`. Returns deterministic AI-ready context; **makes no model call**.

8. Graph APIs

`/api/workspaces/{project}/compliance/graph/frameworks/{frameworkKey}/releases/{releaseCode}` and `.../domains/{domainCode}`, `.../controls/{controlCode}`, `.../requirements/{requirementCode}`.

9. Mapping APIs

`/api/workspaces/{project}/compliance/mappings/...` → `objectives`, `objectives/{objectiveCode}`, `controls/{controlCode}`, `frameworks/compare`, `frameworks/{frameworkKey}/coverage`.

10. Evidence APIs

`/api/workspaces/{project}/compliance/evidence/...` → `types`, `statuses`, `POST context`.

11. Gap APIs

`/api/workspaces/{project}/compliance/gap/...` → `summary`, `domains`, `controls/{controlCode}`, `requirements/{requirementCode}`, `POST context`.

12. Recommendation APIs

/api/workspaces/{project}/compliance/recommendations/... → summary , controls/{controlCode} , requirements/{requirementCode} , POST generate , POST context .

13. Copilot APIs — throttle compliance-copilot

- POST /api/workspaces/{project}/compliance/copilot/message
- GET /api/workspaces/{project}/compliance/copilot/conversations
- GET /api/workspaces/{project}/compliance/copilot/conversations/{conversationUuid}
- POST /api/workspaces/{project}/compliance/copilot/conversations/{conversationUuid}/messages

Example (mock mode):

```
POST /api/workspaces/{project}/compliance/copilot/message
{ "message": "Why is requirement 1-1-1 non-compliant?" }
```

```
{
  "answer": "...cited explanation...",
  "citations": [ { "type": "control", "code": "1-1-1", "source_document_id": "<uuid>" } ],
  "mode": "mock"
}
```

(With AI_COPILOT_DEMO_MODE=true , a demo block with reasoning trace/citations is added. With AI_COPILOT_RAG_ENABLED=true + RAG_ENABLED=true , a rag_context block may be added.)

14. Retrieval / RAG APIs

- **Retrieval (deterministic):** POST /api/workspaces/{project}/compliance/retrieval/query
- **RAG (feature-flagged, returns RAG context only):** POST /api/workspaces/{project}/compliance/rag/query — throttle compliance-rag .

```
POST /api/workspaces/{project}/compliance/rag/query
{ "query": "access control policy" }
```

Returns a bounded, **cited** context package. With RAG disabled, falls back to deterministic retrieval context.

15. Executive APIs (read-only) — throttle compliance-executive

/api/workspaces/{project}/compliance/executive/... → dashboard , scorecard , timeline , explainability , platform . All derived from **real engine data**; no fabricated metrics.

16. Platform AI capability endpoint

GET `/api/ai/platform/capabilities` → modules (adapters), skills, providers, reasoning/retrieval/RAG status, supported contexts, and HUB-wide `module_catalog` (production/reserved/planned).

17. Quenyx AI (Unified AI Workspace — Sprint 20)

RC1.1 naming: this surface is presented in the UI as **Quenyx AI** (the internal/codename remains *Unified AI Workspace*). API paths are unchanged: routes stay under `/api/ai/*` and the SPA routes stay under `/ai-workspace/*` (the branded `/quenyx-ai/*` path redirects to them). No API contract changed in the rename.

Platform-level AI surface beside Dashboard / Workspaces / Integrations. All routes are flat under `/api/ai/*`, Sanctum-protected, throttled by `ai-workspace`, and **scoped by a required workspace UUID** (query string for reads/deletes, request body for writes) — never a numeric id. Every resource id returned is a UUID. Authorization: `ProjectPolicy::accessAi` (any member) for reads, `administerAi` (owner/admin) for provider/permission changes, plus a fine-grained capability check (`can_use_ai`, `can_manage_templates`, `can_manage_providers`, `can_view_costs`, `can_administer`).

Method	Endpoint	Capability	Notes
GET	<code>/api/ai/workspace/summary</code>	access	counts, tokens, flags + caller permissions
GET	<code>/api/ai/conversations</code>	access	list (metadata only)
POST	<code>/api/ai/conversations</code>	use_ai	start a conversation
GET	<code>/api/ai/conversations/{uuid}</code>	access	conversation + messages
POST	<code>/api/ai/conversations/{uuid}/messages</code>	use_ai	runs the shared AI runtime (mock until AI enabled)
GET	<code>/api/ai/activity</code>	access	AI audit timeline
GET	<code>/api/ai/notifications</code>	access	governance events
GET	<code>/api/ai/usage</code>	view_costs	token usage totals/by-provider/daily
GET	<code>/api/ai/costs</code>	view_costs	tokens × configured pricing; <code>pricing_configured</code> flag
GET	<code>/api/ai/skills</code>	access	dynamic skill catalog (Sprint 19)
GET	<code>/api/ai/capabilities</code>	access	full platform capability catalog
GET	<code>/api/ai/prompt-templates</code>	access	list
POST	<code>/api/ai/prompt-templates</code>	manage_templates	create
PUT	<code>/api/ai/prompt-templates/{uuid}</code>	manage_templates	update
DELETE	<code>/api/ai/prompt-templates/{uuid}</code>	manage_templates	delete
GET	<code>/api/ai/providers</code>	access	provider catalog + prefs (secrets never returned)
PUT	<code>/api/ai/providers/{uuid}/settings</code>	manage_providers + admin	encrypted secret write-only; supports <code>clear_secrets</code>
POST	<code>/api/ai/providers/{uuid}/test</code>	manage_providers + admin	real readiness probe (audited); executable providers run <code>health()</code> , others return <code>status: not_executable</code>
GET	<code>/api/ai/permissions</code>	admin	effective per-role matrix
PUT	<code>/api/ai/permissions</code>	admin	upsert per-role overrides

Cost tracking is **derived** from real `ai_conversations` token counts and an optional `config('ai.workspace.pricing')` table; with no pricing configured, responses carry token totals and `pricing_configured=false` with **no fabricated currency**. Conversation message content is stored only when `ai.feature_flags.prompt_logging` is enabled (privacy-preserving default). Provider settings addressing uses a deterministic UUIDv5 (workspace + provider key) so providers are UUID-addressable even before a settings row exists. Provider id `uuid` is also added (additively) to the workspace list (`GET /api/workspaces`) and `ProjectResource` so the UI can pass the workspace UUID.

Provider catalog & execution (RC1.1). `GET /api/ai/providers` returns the declarative provider **catalog** (`App\Services\Ai\AiProviderCatalog`) merged with each workspace's saved preferences. Each entry exposes `label`, `type` (`hosted` / `gateway` / `self_hosted` / `custom` / `dev`), declared `capabilities`, `endpoint`, `docs_url`, `is_default`, `executable`, `platform_configured`, `enabled`, `model`, `secret_configured`, `configured`, and `updated_at`. The catalog covers OpenAI, Anthropic Claude, Google Gemini, Azure OpenAI, OpenRouter, Mistral AI, Cohere, xAI Grok, Ollama, LM Studio, vLLM, LiteLLM Gateway, Hugging Face Inference, and a Custom OpenAI-compatible API. A catalogued provider is **configurable but not executable** until a real adapter exists — today only **OpenAI** has a live execution adapter (`executable: true`); all other catalog entries report `executable: false` and **never fabricate connectivity**. The dev-only `mock` provider is **excluded outside local / testing** and is never the production default. The default provider resolves via `AiProviderRegistry::defaultKey()`: an explicit `AI_PROVIDER` wins, otherwise OpenAI when its key is configured, otherwise `mock` only in local/testing, otherwise `''` (an honest *no provider configured* state, surfaced as `summary.has_provider=false`).

18. QynSight Operations Intelligence (Sprint 21)

Workspace-scoped, **UUID-only** AI surface that turns QynSight monitoring data into explainable operational intelligence. All routes are flat under `/api/qynsight/intelligence/*`, Sanctum-protected, throttled by `ai-workspace`, and **scoped by a required workspace UUID** (query string for reads, request body for writes) — never a numeric id. Every entity id in the path/response is a UUID (a deterministic UUIDv5 derived from internal monitoring ids; no schema change, no numeric ids exposed). These endpoints **reuse the Sprint 20 Quenyx AI runtime** (provider registry, prompt orchestration, conversation service, audit) — **no AI logic is duplicated**.

Authorization (every endpoint): `qynsight` module entitlement + monitoring RBAC (`ProjectPolicy::accessAi`) + the per-workspace AI capability `can_use_ai`. Every AI call is audited, provider-logged, and conversation-logged.

Method	Endpoint	Purpose
GET	<code>/api/qynsight/intelligence/overview?workspace={uuid}</code>	Operations Intelligence dashboard: infrastructure health, open alerts, critical services, top operational risks, predicted capacity risks, recent recommendations, recent AI investigations
POST	<code>/api/qynsight/intelligence/copilot</code>	Monitoring Copilot — grounded Q&A; reuses a Quenyx AI conversation (<code>conversation</code> UUID optional to continue a thread)
GET	<code>/api/qynsight/intelligence/recommendations?workspace={uuid}</code>	Evidence-based operational recommendations
POST	<code>/api/qynsight/intelligence/alerts/{uuid}/explain</code>	Alert explanation: impact, most likely causes, evidence used, related alerts, suggested actions, confidence (evidence-derived only)
POST	<code>/api/qynsight/intelligence/alerts/{uuid}/investigate</code>	Deeper alert investigation (explanation + root cause + timeline)
GET	<code>/api/qynsight/intelligence/incidents/{uuid}/timeline?workspace={uuid}</code>	Deterministic incident timeline from actual event timestamps
POST	<code>/api/qynsight/intelligence/hosts/{uuid}/explain</code>	Host (🔥 Explain) — health, performance, capacity narrative
POST	<code>/api/qynsight/intelligence/services/{uuid}/analyze</code>	Service (🔥 Analyze) — why/what changed/impact/action/related
POST	<code>/api/qynsight/intelligence/capacity/{uuid}/predict</code>	Capacity (🔥 Predict) — trend, forecast, exhaustion, action, risk
POST	<code>/api/qynsight/intelligence/infrastructure/{uuid}/impact</code>	Infrastructure Map (🔥 Impact Analysis) — dependencies, SPOF, blast radius

All write endpoints accept `{ "workspace": "<uuid>", ... }`. The copilot accepts `{ "workspace": "<uuid>", "message": "...", "conversation": "<uuid?>" }` and returns `{ "conversation_uuid", "message_uuid", "answer", "evidence" }`. AI narratives carry an `available` / `ai_enabled` flag; when AI is disabled, a clearly flagged mock narrative is returned while the underlying operational data stays real. **No operational data is fabricated**; when evidence is insufficient the response says so.

19. QynAsset Asset Intelligence + AI Adapter Platform (Sprint 22)

AI Adapter discovery (flat under `/api/ai/*`, Sanctum + `throttle:ai-workspace`, required `workspace` UUID, RBAC `accessAi`, audited, **entitlement-filtered** per workspace):

Method	Endpoint	Description
GET	<code>/api/ai/adapters?workspace={uuid}</code>	Every AI module adapter the workspace is entitled to (metadata, capabilities, entities, skills, providers, actions)
GET	<code>/api/ai/adapters/{module}?workspace={uuid}</code>	One adapter descriptor
GET	<code>/api/ai/adapters/capabilities?workspace={uuid}</code>	Aggregated capabilities across entitled adapters
GET	<code>/api/ai/actions?workspace={uuid}</code>	Aggregated contextual actions across entitled adapters

The AI Workspace builds navigation/actions from these — there is **no hard-coded module list**.

QynAsset Asset Intelligence (flat under `/api/qynasset/intelligence/*`, Sanctum + `throttle:ai-workspace`, `qynasset` entitlement, RBAC `accessAi`, `can_use_ai` for AI actions, UUID-only):

Method	Endpoint	Description
GET	<code>/api/qynasset/intelligence/overview?workspace={uuid}</code>	Asset Intelligence dashboard: inventory summary, discovery, capacity, recommendations, recent AI investigations
POST	<code>/api/qynasset/intelligence/copilot</code>	Asset Copilot — grounded Q&A; reuses a Quenyx AI conversation
GET	<code>/api/qynasset/intelligence/recommendations?workspace={uuid}</code>	Evidence-based asset recommendations
POST	<code>/api/qynasset/intelligence/assets/{uuid}/explain</code>	Asset (🔥 Explain) — discovery, activity, hardware, gaps
POST	<code>/api/qynasset/intelligence/assets/{uuid}/dependencies</code>	Dependency (🔥 Analyze) — depends on / serves, neighbors
POST	<code>/api/qynasset/intelligence/assets/{uuid}/lifecycle</code>	Lifecycle (🔥 Forecast) — replacement priority, business impact (warranty/EOL not collected)
POST	<code>/api/qynasset/intelligence/assets/{uuid}/impact</code>	Relationship (🔥 Impact) — blast radius / SPOF
POST	<code>/api/qynasset/intelligence/licenses/review</code>	License (🔥 Review) — honest "not collected" until an inventory/license integration exists

Asset UUIDs are deterministic UUIDv5 (`AssetEntityId`) over the discovered host; **no numeric ids are exposed**. Inventory, hardware, lifecycle, and license facts are **real or honestly absent** — never fabricated.

20. Notes on examples

Example request/response shapes are representative of the controllers' contracts. For exact current fields, call the endpoint against a seeded workspace, or read the corresponding controller/resource in `app/Http/Controllers/**`. Do not assume undocumented fields.